

SCGC Homework - Omar Mahmoud

Task 0:

This task was only about preparation of the VM and resizing the disk and filesystem

Task 1:

1.1 Creation of the kubernetes cluster and verification of its creation

```
[student@lab-kubernetes:~]$ kind create cluster
Creating cluster "kind" ...
  ✓ Ensuring node image (kindest/node:v1.35.0) 📦
  ✓ Preparing nodes 📦
  ✓ Writing configuration 📄
  ✓ Starting control-plane 🚦
  ✓ Installing CNI 🚦
  ✓ Installing StorageClass 🗄️
Set kubectl context to "kind-kind"
You can now use your cluster with:

kubectl cluster-info --context kind-kind

Not sure what to do next? 😅 Check out https://kind.sigs.k8s.io/docs/user/quick-start/
[student@lab-kubernetes:~]$ kubectl get nodes
NAME                 STATUS    ROLES    AGE   VERSION
kind-control-plane   NotReady  control-plane   19s   v1.35.0
[student@lab-kubernetes:~]$
```

1.2 Creation of the HTML, deployment, and config files

```
[student@lab-kubernetes:~]$ nano nginx-html.yaml
[student@lab-kubernetes:~]$ cat nginx-html.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-html
data:
  index.html: |
    <html><body>Task 1 homework</body></html>
[student@lab-kubernetes:~]$ kubectl apply -f nginx-html.yaml
configmap/nginx-html created
```

The config contains the nginx server configurations including the ports used for the html and the metrics

```

[student@lab-kubernetes:~$ nano nginx-config.yaml
[student@lab-kubernetes:~$ kubectl apply -f nginx-config.yaml
configmap/nginx-conf created
[student@lab-kubernetes:~$ cat nginx-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  default.conf: |
    server {
      listen 80;
      server_name localhost;

      location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
      }

      server {
        listen 8080;
        server_name localhost;

        location /metrics {
          stub_status;
        }
      }
    }
  
```

The deployment includes the metadata and specs for running the application

```

[student@lab-kubernetes:~$ nano nginx-deployment.yaml
[student@lab-kubernetes:~$ cat nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: gitlab.cs.pub.ro:5050/scgc/cloud-courses/nginx:latest
          ports:
            - containerPort: 80
            - containerPort: 8080
          volumeMounts:
            - name: nginx-html-vol
              mountPath: "/usr/share/nginx/html/index.html"
              subPath: "index.html"
            - name: nginx-conf-vol
              mountPath: "/etc/nginx/conf.d/default.conf"
              subPath: "default.conf"
      volumes:
        - name: nginx-html-vol
          configMap:
            name: nginx-html
            items:
              - key: "index.html"
                path: "index.html"
        - name: nginx-conf-vol
          configMap:
            name: nginx-conf
            items:
              - key: "default.conf"
                path: "default.conf"
[student@lab-kubernetes:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx created
[student@lab-kubernetes:~$
  
```

```

[student@lab-kubernetes:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-77cc998945-6q4bw             1/1     Running   0           42s
[student@lab-kubernetes:~$ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1             1           48s
  
```

1.3 Creation of the nginx survive so that the html and the metrics are exposed externally

```
[student@lab-kubernetes:~$ nano nginx-service.yaml
[student@lab-kubernetes:~$ cat nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
    - name: metrics
      protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 30088
[student@lab-kubernetes:~$ kubectl apply -f nginx-service.yaml
service/nginx created
[student@lab-kubernetes:~$ ]
```

Task2:

2.1 Creation of the prometheus exporter deployment with the url to be monitored and which internal port to be exposed

```
[student@lab-kubernetes:~$ cat promeporter-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: promexporter
  labels:
    app: promexporter
spec:
  replicas: 1
  selector:
    matchLabels:
      app: promexporter
  template:
    metadata:
      labels:
        app: promexporter
    spec:
      containers:
        - name: nginx-prometheus-exporter
          image: nginx/nginx-prometheus-exporter:1.5.1
          args:
            - --nginx.scrape-uri=http://nginx:8080/metrics
          ports:
            - containerPort: 9113
[student@lab-kubernetes:~$ kubectl apply -f promeporter-deployment.yaml
deployment.apps/promexporter created
[student@lab-kubernetes:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
nginx-77cc998945-6q4bw              1/1      Running   0           66m
promexporter-7d85bb5cf6-qfc62       1/1      Running   0           18s
[student@lab-kubernetes:~$ kubectl get deployments
NAME            READY    UP-TO-DATE    AVAILABLE    AGE
nginx           1/1      1              1            66m
promexporter    1/1      1              1            23s
[student@lab-kubernetes:~$ ]
```

2.2 Creation of the prometheus service so that the exporter can be exposed externally

```

student@lab-kubernetes:~$ kubectl apply -f promexporter-service.yaml
service/promexporter created
student@lab-kubernetes:~$ kubectl get services

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
nginx	NodePort	10.96.182.193	<none>	80:30080/TCP,8080:30088/TCP
promexporter	ClusterIP	10.96.124.72	<none>	9113/TCP

```

student@lab-kubernetes:~$ cat promexporter-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: promexporter
spec:
  type: ClusterIP
  selector:
    app: promexporter
  ports:
    - protocol: TCP
      port: 9113
      targetPort: 9113
student@lab-kubernetes:~$

```

Task 3:

3.1 Installation of heml and adding the prometheus repo then creating the monitoring namespace(This screenshot was taken after completion of all tasks which is why it includes grafana from the next task)

```

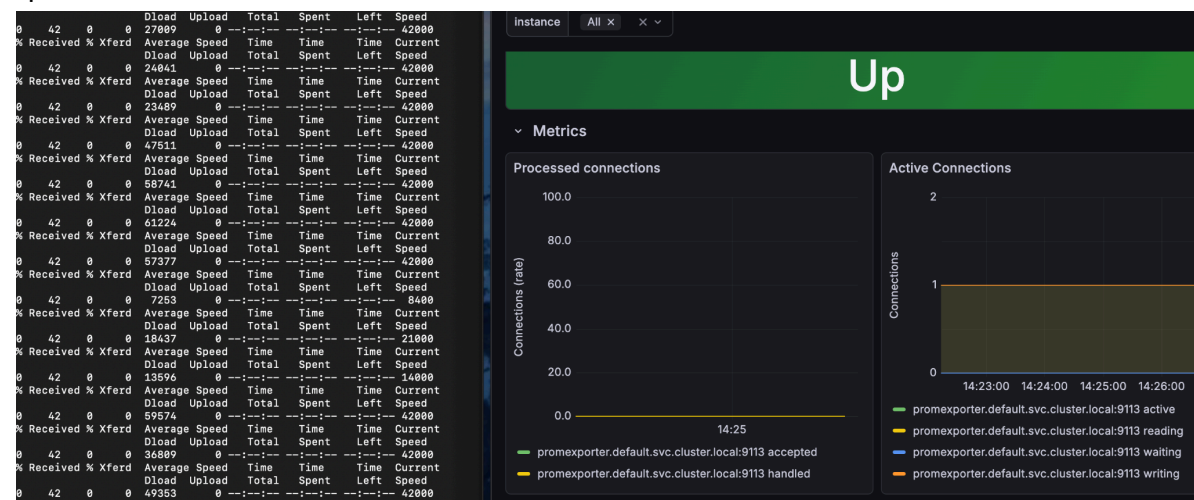
student@lab-kubernetes:~$ kubectl get pods -n monitoring

```

NAME	READY	STATUS	RESTARTS
grafana-dfbb9ccd-mbv6b	1/1	Running	0
prometheus-alertmanager-0	1/1	Running	0
prometheus-kube-state-metrics-56c55854b6-xn6xk	1/1	Running	1 (42h ago)
prometheus-prometheus-node-exporter-sn5jp	1/1	Running	0
prometheus-prometheus-pushgateway-75b8bf7c77-tqjvt	1/1	Running	0
prometheus-server-7c8867679f-94ml9	2/2	Running	0

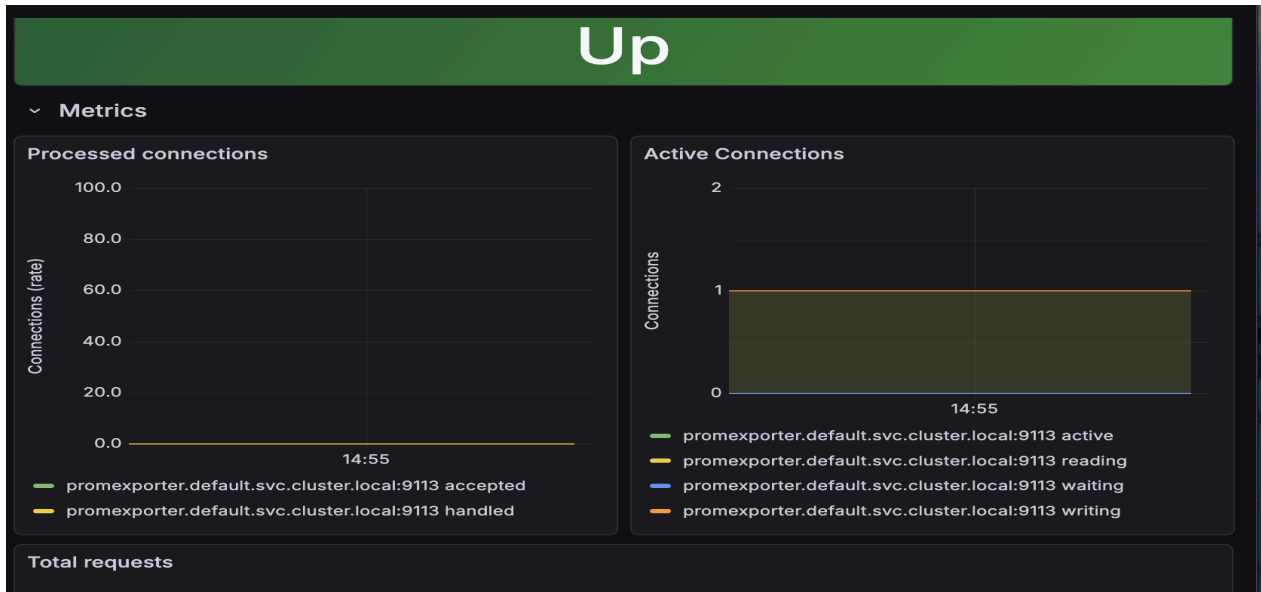
The figure is a screenshot of a Prometheus server interface. The left pane shows the raw Prometheus query results for the query: `student1@lab-kubeznetes:~$ for i in {1..20}; do curl http://172.18.0.2:30080; done`. The right pane shows a time series graph of the results. The y-axis represents the value of the metric, ranging from 40.00 to 85.00. The x-axis represents time, ranging from 08:02:30 to 08:03:20. The graph shows a sharp increase in the value from approximately 40.00 to 85.00 around 08:03:10.

- 4.1 Added the grafana helm repo then deployed it in the monitoring namespace as seen above
- 4.2 Port forwarded grafana like prometheus and configured prometheus to be a dataset for grafana
- 4.3 imported the nginx exporter dashboard and generated traffic to verify that the dashboard updates

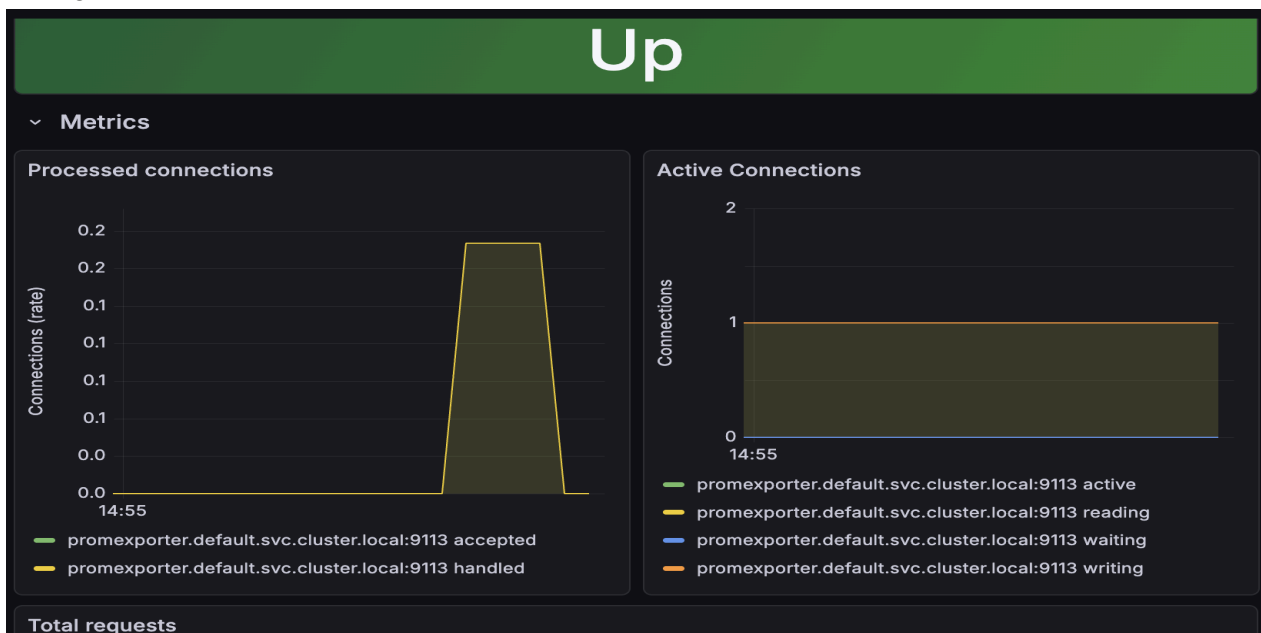


- Using `slowhttptest` to observe the behavior of the dashboard before,during, and after a DoS attack.

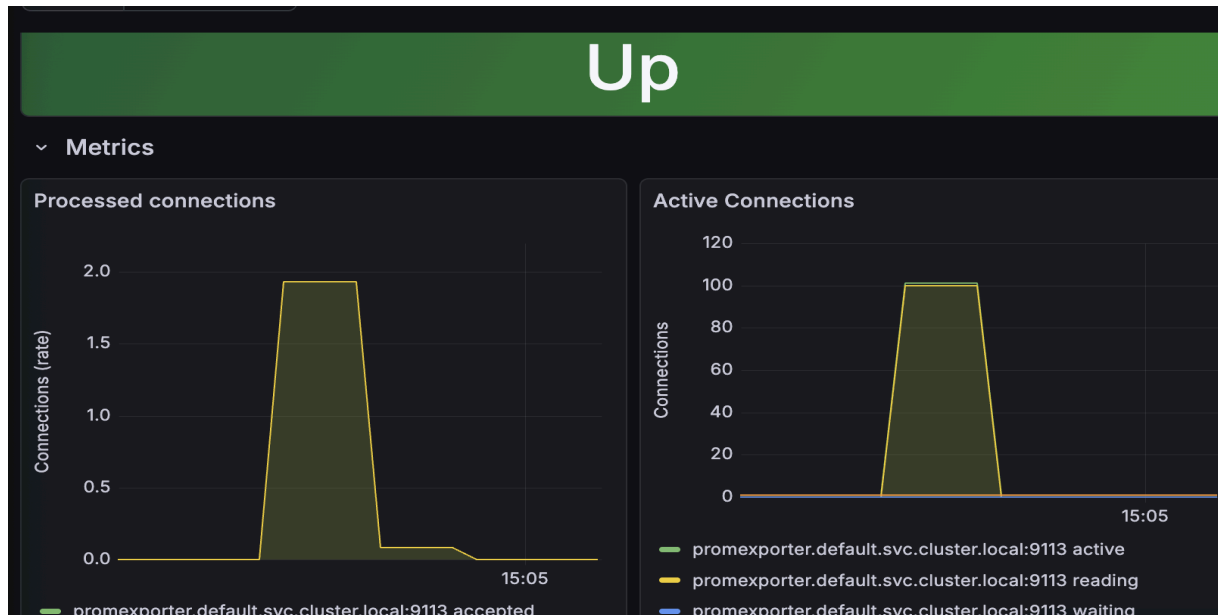
Before attack:



During attack:



After Attack:



Task 5 Conclusions: before the attack the active connections in the grafana dashboard were low and stable, which is the normal and expected result, but during the attack the active connections and processed connection spiked indicated a high load which is a behavior specific with DoS attacks, when the attack stopped, the connections dropped significantly reaching the same stable levels they are at before the attack, this is an ideal use of promethrus and grafana for detecting abnormal behaviors in systems.

Task 6:

To improve the security of the infrastructure used in this homework, the following actions should be taken:

- Use HTTPS instead of HTTP so traffic is encrypted and cannot be intercepted
- Add rate limiting in nginx to prevent too many requests from one client
- Add authentication for prometheus and grafana and use access control in kubernetes
- Use firewall rules to allow only trusted IP addresses
- Use resource limits for CPU and memory so one service cannot exhaust the system